

EXERCISE I

In reference to the Stochastic Gradient Descent (SDG) algorithm

I.a) Explain the concept of loss function and provide one example

The loss function is the function that computes the distance between the current output of the algorithm and the expected output. It's a method to evaluate how your algorithm models the data. It can be categorized into two groups. One for classification (discrete values, 0,1,2...) and the other for regression (continuous values).

example: Mean squared error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

Where:

- n is the number of data points.
- $\hat{Y}_i$ is the predicted value for the i -th observation.
- Y_i is the true value for the i -th observation.

I.b) Does the gradient descent algorithm guarantee global minimization of the loss function? Motivate your answer

the guarantee of achieving global minimization of the loss function depends on multiple factors:

- **Convexity of the Loss Function:** If the loss function is convex, then gradient descent can indeed converge to the global minimum. Convex problems have the property that any local minimum is also a global minimum, making gradient descent an effective method for finding the global optimum.
- **Non-Convex Loss Functions:** gradient descent does not guarantee finding the global minimum. Instead, it may converge to a local minimum or a saddle point. The algorithm's convergence depends on factors like the initialization of parameters and the learning rate.
- **Learning Rate:** the choice of the learning rate can significantly affect the algorithm's performance. A learning rate that's too high can cause the algorithm to overshoot minima, while a too low learning rate can result in slow convergence. Adaptive learning rate algorithms like AdaGrad, RMSprop, and Adam have been developed to mitigate some of these issues by adjusting the learning rate during training.

I.c) Assume you train a neural network for 1000 epochs. Backpropagation requires (0.001*Nsample +0.2) seconds per epoch. You have a training set with 15.000 samples

and a sampling ratio of 15% for the stochastic selection of the samples. Compute total training time.

epochs = 1000 # Total epochs

N_samples = 15000 # Total number of samples

sampling_ratio = 0.15 # Sampling ratio for stochastic selection

$\text{samples_per_epoch} = N_samples * \text{sampling_ratio} = 15000 * 0.15 = 2250$

$\text{time_per_epoch} = \text{samples_per_epoch} * 0.001 + 0.2 \text{ [s]} = 2.45 \text{ s}$

Computing the total training time

$\text{total_time} = \text{time_per_epoch} * \text{epochs} = 2.45\text{s} * 1000 = 2450 \text{ s}$

EXERCISE II

Consider a convolutional neural network layer with the following characteristics: 64 input channels (40x40), convolutional filters size 5x5, 256 output channels. No padding and stride 1.

II.a) Compute the total storage size needed for input, output activations and weights assuming activations are 16bit and weights are 8bit.

Input		Kernel		Output	
W_in	40	K (W)	5	W_out	$[(W_{in}-K+2P)/S]+1$ 36
H_in	40	K (H)	5	H_out	$[(H_{in}-K+2P)/S]+1$ 36
C_in	64	C_in	64	C_out	256
		C_out	256		
		S_w	1		
		S_h	1		
		padding	P	0	

Memory occupation			
Input	Kernel	Output	TOT size
W_in*H_in*C_in = 102,400	K*K*C_in*C_out = 409,600	W_out*H_out*C_out = 331,776	In+K+Out 843,776

Input = 102,400 * 2Byte = 204 800 B

Kernel = 409 600 B

Output = 331,776 * 2Byte= 663 552 B

Total = 1277952 B

II.b) Compute the total number of MAC operations needed to compute all the output channels in the layer.

MACS		
MAC (1_in_ch, 1_out_ch) =	K*K*H_out*W_out =	32400
MAC (all_out_ch) =	K*K*C_in*H_out*W_out*Ch_out =	530841600

II.c) Compute the number of SUM and the number of DIV operations needed to perform average pooling (3x3) on all the output channels, consider pooling stride = 3.

AVG POOL					
Input		AVG Pool		Output	
W_out	36	K (W)	3	W_out	12
H_out	36	K (H)	3	H_out	12
C_out	256	stride	3	C_out	256
		pad	0		

SUM operations			DIV operations		
kernel only	K*K-1	8	kernel only	-----	1
all output	(K*K-1)*Wout*Hout* Cout	294912 SUM	all output	(1)*Wout*Hout* Cout	36864 DIV
TOT		294912 SUM			36864 DIV

EXERCISE III

A parallel cluster of RISC-V processors is executing a convolutional layer with 9 input channels and 18 output channels. The input spatial size is 64x64. Inputs, outputs, and weights are quantized to 16 bits. The convolutional filter size is 3x3, no padding is used. RELU non-linearity is applied in place on the output. Assume that on average a RISC-V processor can perform a MAC in 4 cycles and a RELU operation in 6 cycles. RELU and convolutions are parallelized in an ideal fashion, with no overhead and perfect load balancing.

III.a) Compute the L1 minimal shared memory size that is needed to store the complete layer (input and output channels as well as weights) in L1

Memory occupation			Precision	2B
Input	Kernel	Output	TOT size	
$W_{in} \times H_{in} \times C_{in} =$	$K \times K \times C_{in} \times C_{out} =$	$W_{out} \times H_{out} \times C_{out} =$	$(In+K+Out) \times \text{precision}$	
36,864	1,458	69,192	215028 B	

III.b) Compute the total number of cycles to process the layer on a 4-core cluster

	cycles	Processed by
MAC	4 cycles	4 Cores
ReLU	6 cycles	4 Core

$$\begin{aligned}
 \text{Cycles_conv (4 cores)} &= (H_{out} \times W_{out} \times C_{in} \times (C_{out}/N_{cores}) \times K \times K) \times [\text{MAC}] \text{ cycles} = 5604552 \text{ cycles} \\
 \text{Cycles_ReLU (4 core)} &= ((C_{out}/N_{cores}) \times W_{out} \times H_{out}) \times [\text{ReLU}] \text{ cycles} = \\
 &= (19/N_{cores} \times 126 \times 126) \times [\text{ReLU}] \text{ cycles} = 103788 \text{ cycles} \\
 \text{Cycles_tot} &= \mathbf{5708340 \text{ cycles}}
 \end{aligned}$$

Note: if the parallelization was not ideal, then:

$$\begin{aligned}
 \text{Cycles_conv (4 cores)} &= (H_{out} \times W_{out} \times C_{in} \times \text{ceil}(C_{out}/N_{cores}) \times K \times K) \times [\text{MAC}] \text{ cycles} = 6227280 \text{ cycles} \\
 \text{Cycles_ReLU (4 core)} &= (\text{ceil}(C_{out}/N_{cores}) \times W_{out} \times H_{out}) \times [\text{ReLU}] \text{ cycles} = \\
 &= (19/N_{cores} \times 126 \times 126) \times [\text{ReLU}] \text{ cycles} = 115320 \text{ cycles} \\
 \text{Cycles_tot} &= \mathbf{6342600 \text{ cycles}}
 \end{aligned}$$

III.c) Compute the speedup with respect to sequential when the cluster has 8 cores

Sequential:

	cycles	Processed by
MAC	4 cycles	1 Cores
ReLU	6 cycles	1 Core

$$\begin{aligned}
 \text{Cycles_conv (4 cores)} &= (H_{out} \times W_{out} \times C_{in} \times (C_{out}/N_{cores}) \times K \times K) \times [\text{MAC}] \text{ cycles} = 22418208 \text{ cycles} \\
 \text{Cycles_ReLU (4 core)} &= ((C_{out}/N_{cores}) \times W_{out} \times H_{out}) \times 3 \text{ cycles} = \\
 &= (19/N_{cores} \times 126 \times 126) \times [\text{ReLU}] \text{ cycles} = 415152 \text{ cycles} \\
 \text{Cycles_tot} &= \mathbf{22833360 \text{ cycles}}
 \end{aligned}$$

8 cores:

	cycles	Processed by	
MAC	4 cycles	8	Cores
ReLU	6 cycles	8	Core

Cycles_conv (4 cores) = (Hout * Wout * Cin * (Cout/Ncores) * K * K) * [MAC] cycles = 2802276 cycles

Cycles_ReLU (4 core) = ((Cout/N_cores) *Wout*Hout) * 3 cycles =
= (19/N_cores*126*126)* [ReLU] cycles = 51894 cycles

Cycles_tot = **2854170 cycles**

Speedup= 22833360 /2854170 = 8x